



آزمایشگاه طراحی سیستم‌های دیجیتال

ترم تابستان ۱۴۰۵

استاد: دکتر انصاری

آزمایش اول

تشخیصی مضرب ۳ و ۱۱ در عدد BCD

معین آعلی

خلاصه

در این آزمایش یک مدار ترکیبی محض (بدون فلیپ‌فلاپ، بدون کلاک، بدون ماشین‌حالت) طراحی و در نرم‌افزار Logisim Evolution پیاده‌سازی شده که یک عدد BCD چهاررقمی $D_3D_2D_1D_0$ (هر رقم ۴ بیت، مقدار کل بین ۰۰۰۰ تا ۹۹۹۹) را می‌گیرد و یک خروجی تک‌بیتی می‌دهد: ۱ اگر عدد مضربی از ۳ باشد (مدار `mod3_checker`)، و به همان شکل برای مضرب ۱۱ (مدار `mod11_checker`). طبق متن آزمایش «در طراحی این مدار فقط مجاز به استفاده از گیت‌های پایه هستید»، در کل مدار فقط از گیت‌های AND، OR، XOR، NOT استفاده شده — نه از تراشه، نه از قطعه‌ی آماده‌ی `Comparator/Adder` کتابخانه‌ی Arithmetic. مدار به‌صورت کاملاً سلسله‌مراتبی و از پایین به بالا ساخته شده است: یک تمام‌جمع‌کننده‌ی ۱ بیتی (`full_adder`)، جمع‌کننده‌های ۴ و ۵ بیتی ripple-carry ساخته‌شده از آن (`adder4`، `adder5`)، و دو شبکه‌ی مقایسه‌ی کوچک (`eq5`، `cmp3`) که با ترکیب این بلوک‌ها، مدارهای اصلی `mod3_checker` و `mod11_checker` را می‌سازند.

هدف آزمایش

- آشنایی عملی با طراحی یک مدار ترکیبی در محیط شماتیک Logisim، با محدودیت استفاده‌ی صرف از گیت‌های پایه.
- تمرین طراحی سلسله‌مراتبی: ساخت یک بلوک ساده (تمام‌جمع‌کننده) و استفاده‌ی مکرر از آن برای ساخت بلوک‌های بزرگ‌تر.
- یافتن یک راه‌حل ترکیبی (نه ترتیبی) برای مسئله‌ی تشخیص باقیمانده — با استفاده از خواص عددی مبنای ۱۰ به‌جای شمارنده یا ماشین‌حالت.
- صحت‌سنجی جامع (نه فقط چند نمونه) با اجرای بردار تست روی تمام فضای ورودی معتبر.

چرا اصلاً می‌شود این را بدون FSM حل کرد

چون مدار باید ترکیبی محض باشد، به‌جای باز کردن یک ماشین‌حالت «باقیمانده $\text{mod } 3 / \text{mod } 11$ » روی ۱۶ بیت ورودی (که یک مدار ترتیبی یا لااقل بسیار پیچیده می‌طلبد)، از دو اتحاد عددی مربوط به مبنای ۱۰ استفاده شده که مسئله را به یک جمع رقم‌ها و یک مقایسه‌ی کوچک تبدیل می‌کند. هر دو اتحاد پیش از ساخت حتی یک گیت، با یک اسکریپت پایتون روی هر ۱۰۰۰۰ مقدار BCD به‌صورت brute-force راستی‌آزمایی شدند (صفر مورد نقض).

مضرب ۳ — جمع ارقام. چون $10 \equiv 1 \pmod{3}$ ، هر توان ۱۰ هم به پیمانه‌ی ۳ برابر ۱ است، پس

$$N = 1000D_3 + 100D_2 + 10D_1 + D_0 \equiv D_3 + D_2 + D_1 + D_0 \pmod{3}.$$

کافی است رقم‌ها را جمع بزنیم (S ، بین ۰ تا ۳۶، در ۶ بیت جا می‌شود) و ببینیم S مضرب ۳ هست یا نه. برای چک‌کردن خود S بدون شمارنده، از وزن دودویی استفاده می‌شود: چون $10 \equiv -1 \pmod{3}$ ، وزن بیت k م برابر

$3 \bmod 2^k$ است که بین 1 (بیت‌های زوج s_0, s_2, s_4) و -1 (بیت‌های فرد s_1, s_3, s_5) نوسان می‌کند. با تعریف

$$P = s_0 + s_2 + s_4 \in \{0, 1, 2, 3\}, \quad Q = s_1 + s_3 + s_5 \in \{0, 1, 2, 3\},$$

داریم $S \equiv 0$ پس شرط $S \equiv P - Q \pmod{3}$ می‌شود $P \bmod 3 = Q \bmod 3$ ، یعنی یا $P = Q$ ، یا $\{P, Q\} = \{0, 3\}$ (چون $3 \equiv 0$). جالب اینکه محاسبه‌ی خود P و Q (جمع سه بیت تکی) چیزی نیست جز یک تمام‌جمع‌کننده‌ی دیگر: full_adder(s_0, s_2, s_4) مستقیماً $\{p_1, p_0\}$ را می‌دهد.

مضرب ۱۱ - جمع متناوب ارقام. چون $10 \equiv -1 \pmod{11}$:

$$N \equiv -D_3 + D_2 - D_1 + D_0 = (D_0 + D_2) - (D_1 + D_3) \pmod{11}.$$

با $A = D_0 + D_2$ و $B = D_1 + D_3$ (هرکدام بین 0 تا 18)، بازه‌ی $A - B$ بین -18 تا 18 است؛ تنها مضرب‌های 11 در این بازه $11, 0, -11$ هستند. پس

$$N \equiv 0 \pmod{11} \iff A + 11 = B \text{ یا } A = B \text{ یا } A = B + 11.$$

سه مقایسه‌ی تساوی که با جمع‌کننده (برای 11 و $B + 11$) و کامپریتر تساوی پیاده می‌شوند - بدون هیچ کم‌کننده یا مکمل‌دوی صریح.

تمام‌جمع‌کننده‌ی ۱ بیتی (full_adder)

پایه	جهت	پهنا
C_{in}, B, A	ورودی	۱ بیت
C_{out}, S	خروجی	۱ بیت

مدار با ۲ گیت XOR، ۲ گیت AND و ۱ گیت OR ساخته شده – بلوک پایه‌ی تمام محاسبات این آزمایش:

• $X_1 = A \oplus B$ – نیمه‌جمع، بدون در نظر گرفتن رقم نقلی ورودی.

• $P_1 = A \wedge B$ – آیا از جمع A و B به‌تنهایی رقم نقلی تولید می‌شود؟

• $S = X_1 \oplus C_{in}$ – جمع نهایی سه ورودی.

• $P_2 = X_1 \wedge C_{in}$ – آیا از جمع X_1 و C_{in} رقم نقلی تولید می‌شود؟

• $C_{out} = P_1 \vee P_2$ – رقم نقلی خروجی نهایی.

PLACEHOLDER

Replace this file with a Logisim screenshot of: full_adder circuit
(Zoom to Fit, then File > Export Image, or a window screenshot of the canvas)

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

بلوک‌های سطح بالاتر

بلوک	ورودی → خروجی	ساخته شده از	گیت
adder4	$A[4], B[4] \rightarrow S[5]$	۴ نمونه‌ی full_adder زنجیره‌ای (ripple)، رقم نقلی بیت صفر به GND	$4 \times 5 = 20$
adder5	$A[5], B[5] \rightarrow S[6]$	۵ نمونه‌ی full_adder زنجیره‌ای	$5 \times 5 = 25$
cmp3	$p_8, p_0, q_8, q_0 \rightarrow \text{Eq}$	شبکه‌ی $P \bmod 3 = Q \bmod 3$ (فرمول بالا)	$2+6+7+2 = 17$
eq5	$A[5], B[5] \rightarrow \text{Eq}$	$(A_i \oplus B_i) \text{NOT} \times 5$ سپس درخت AND	$5+5+4 = 14$

cmp3 از ۲ گیت XOR، ۶ گیت NOT و ۷ گیت AND و ۲ گیت OR ساخته شده: ابتدا $p_0=q_0$ و $p_8=q_8$ با XOR+NOT چک می‌شوند، همزمان چهار ترکیب $P=0, P=3, Q=0, Q=3$ با NOT+AND محاسبه می‌شوند، و در نهایت سه شرط $\{P=Q\}$ ، $\{P=0 \wedge Q=3\}$ ، $\{P=3 \wedge Q=0\}$ با دو گیت OR ترکیب می‌شوند. eq5 برای هر بیت یک XOR و یک NOT (یعنی XNOR با گیت‌های پایه) می‌سازد و در نهایت ۵ نتیجه را با درخت AND ترکیب می‌کند.

معماری mod3_checker

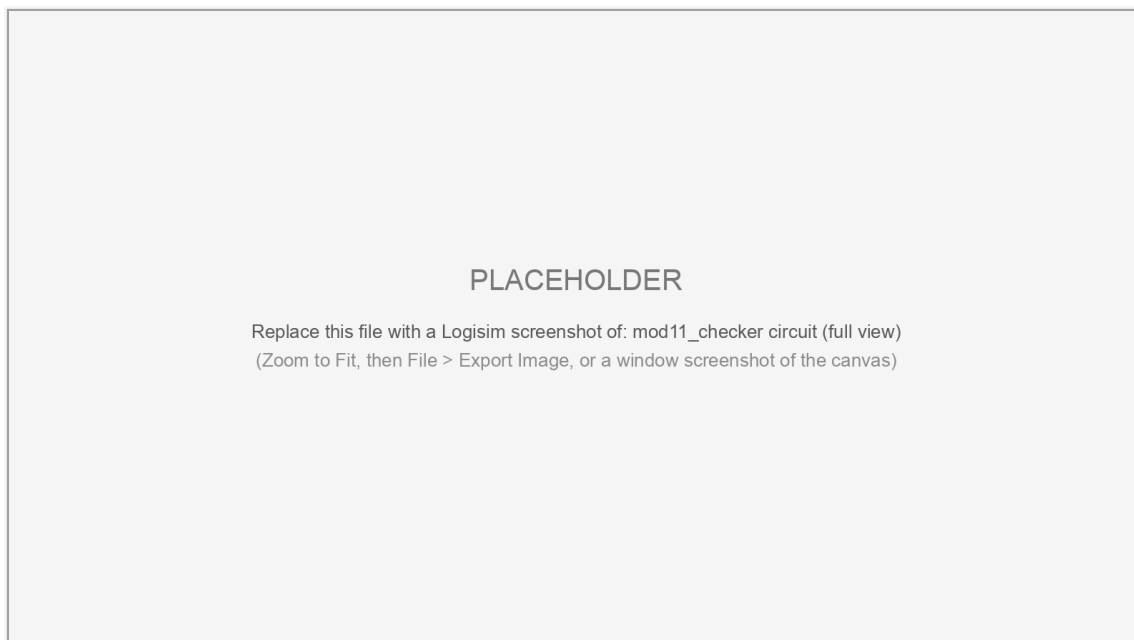
پایه	جهت	پهنا
D3, D2, D1, D0	ورودی	۴ بیت (رقم BCD)
out	خروجی	۱ بیت

PLACEHOLDER

Replace this file with a Logisim screenshot of: mod3_checker circuit (full view)
(Zoom to Fit, then File > Export Image, or a window screenshot of the canvas)

D_0, D_1 با adder4 جمع می‌شوند و D_2, D_3 نیز جدا؛ دو مجموع ۵ بیتی با adder5 به مجموع کل $S = D_0 + D_1 + D_2 + D_3$ (۶ بیت) تبدیل می‌شوند. سپس بیت‌های زوج S با یک full_adder جمع می‌شوند ($\rightarrow P$) و بیت‌های فرد با یکی دیگر ($\rightarrow Q$)، و cmp3 شرط $P \bmod 3 = Q \bmod 3$ را چک می‌کند. جمع کل گیت خام: $2 \times 20 + 25 + 2 \times 5 + 17 = 92$ گیت.

معماری mod11_checker



Constant ۵ قطعه‌ی $A = D_0 + D_2$ و $B = D_1 + D_3$ با دو adder4 حساب می‌شوند؛ عدد ثابت ۱۱ (01011_2) با ۵ قطعه‌ی $A = B$ و $A = BP_{11}$ را چک می‌کنند و دو گیت OR نهایی، خروجی سه مقایسه را OR می‌کنند. جمع تک‌بیتی مستقیماً به ورودی B هر adder5 وصل است تا $A+11$ و $B+11$ به دست بیایند. سه eq5 تساوی‌های کل گیت خام: $2 = 136 + 2 + 3 \times 14 + 2 \times 25 + 2 \times 20$ گیت.

ردیابی دستی (برای ارائه)

مثال ۱-۱۳۲ $N = 132$ (هم مضرب ۳ است، هم مضرب ۱۱): $D_3 D_2 D_1 D_0 = 0132$.

گام	mod3_checker	mod11_checker
جمع	$S = 2+3+1+0 = 6 = 00110_2$	$B = D_1 + D_3 = 3, A = D_0 + D_2 = 3$
کاهش	$P = s_0 + s_2 + s_4 = 0+1+0 = 1$ $Q = s_1 + s_3 + s_5 = 1+0+0 = 1$	$A = B \Rightarrow EQ2 = 1$
مقایسه	$P = Q = 1 \Rightarrow cmp3 = 1$	$out = EQ1 \vee EQ2 \vee (EQ1 = EQ3 = 0, EQ3 = 1)$
نتیجه	$(132/3=44) \checkmark out = 1$	$(132/11=12) \checkmark out = 1$

مثال ۲-۱۳۳ $N = 133$ (نه مضرب ۳، نه مضرب ۱۱): $D_3 D_2 D_1 D_0 = 0133$.

گام	mod3_checker	mod11_checker
جمع	$S = 3+3+1+0 = 7 = 00111_2$	$B = 3+0 = 3, A = 3+1 = 4$
کاهش	$Q = 1+0+0 = 1, P = 1+1+0 = 2$	$B+11 = 14 \neq 4, A+11 = 15 \neq 3, A \neq B$
نتیجه	$\checkmark P=2 \neq Q=1 \Rightarrow out=0$	همه‌ی سه تساوی صفر $\Rightarrow out=0$

چیدمان و سیم‌کشی

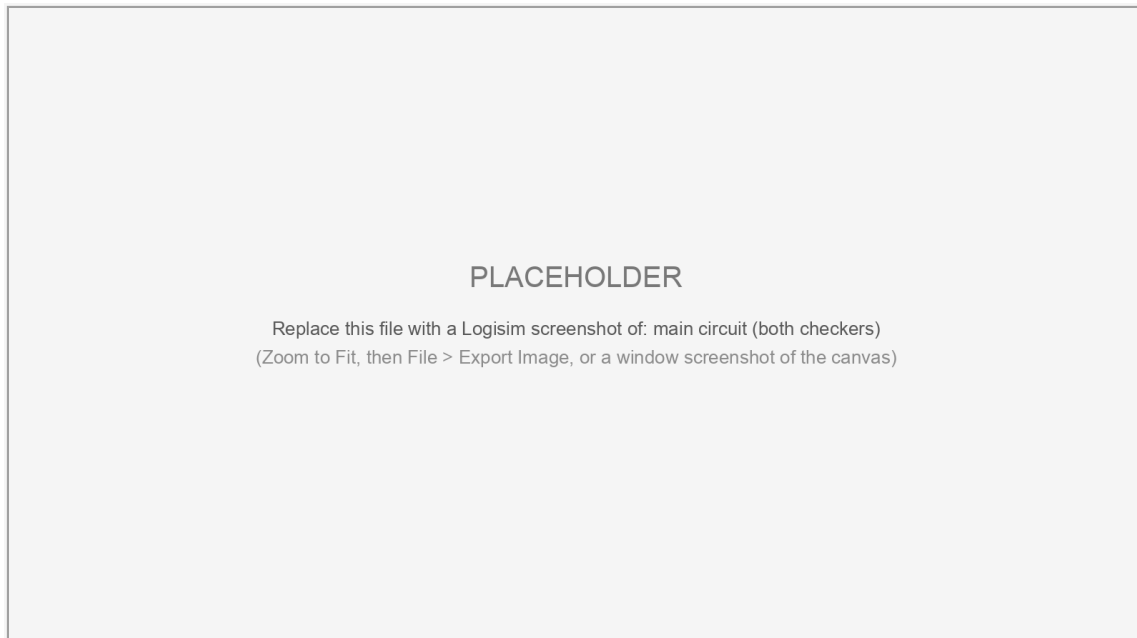
هر بلوک یک مدار مستقل با یک ظاهر (appearance) سفارشی است — یعنی وقتی داخل مدار بالادستی جای می‌گیرد، فقط یک جعبه‌ی کوچک با پین‌های لیبل‌دار دیده می‌شود، نه انبوه گیت داخلش. نکات چیدمانی:

- هر رقم ورودی (پایه‌ی ۴ بیتی) بلافاصله با یک تفکیک‌کننده به ۴ بیت تکی می‌شکند و هر بیت با یک سیم کوتاه به یک تونل نام‌دار می‌رود؛ از همان‌جا به بعد هیچ باس چندبیتی داخلی نداریم — تمام محاسبات با سیم‌های تک‌بیتی انجام می‌شود.
- هر اتصال «راه‌دور» (بین دو بلوک، یا بین دو طبقه‌ی گیت داخل eq5/cmp3) از یک تونل هم‌نام رد می‌شود، نه یک سیم بلند؛ هر تونل با یک ساقه‌ی کوتاه (۳۰ واحد) به پایه‌ی واقعی‌اش وصل است. این یعنی هیچ سیمی مسیر بلندی طی نمی‌کند و هیچ دو سیمی که به هم ربطی ندارند از کنار هم رد نمی‌شوند.
- طبقات full_adder داخل adder5/adder4 هر کدام ۳۰۰ واحد از هم فاصله دارند (در برابر جعبه‌ی 80×80 خود full_adder) — فاصله‌ی خیلی بیشتر از حد لازم. همین الگو در eq5/cmp3 (۱۵۰–۲۰۰ واحد بین گیت‌های یک ستون، ۳۰۰+ واحد بین ستون‌ها) و در mod11_checker/mod3_checker (۳۰۰–۶۰۰ واحد بین بلوک‌ها) تکرار شده.
- هر یک از ۸ مدار فایل (full_adder, adder4, adder5, cmp3, eq5, mod3_checker, mod11_checker, main) با یک بررسی‌گر هندسی خودکار چک شده — هم‌پوشانی هم‌راستای دو سیم از دو نت متفاوت، تماس نقطه‌ای انتهایی یک سیم با بدنه‌ی سیم دیگر، و برخورد جعبه‌ی محیطی قطعات. روی هر ۸ مدار صفر مشکل گزارش شد.

فایل circ. با اسکریپت build_circ.py به‌صورت برنامه‌نویسی‌شده تولید می‌شود (نه با کشیدن دستی در محیط گرافیکی)، دقیقاً به همین دلیل که فاصله‌ها و مسیر سیم‌ها همیشه تحت کنترل و قابل بازتولید بمانند.

مدار main

برای مشاهده‌ی هم‌زمان هر دو خروجی، مدار main دو بلوک mod3_checker و mod11_checker را روی همان ۴ پایه‌ی ورودی مشترک $D_3 \dots D_0$ سوار می‌کند و دو خروجی mult_of_3 و mult_of_11 را جدا بیرون می‌دهد:



نتایج تست

هر ۸ مدار با بردار تست و از طریق خط فرمان Logisim در حالت headless (-test-vector) تست شدند. full_adder، mod11_checker/mod3_checker و eq5، cmp3، adder5، adder4 روی تمام فضای ورودی‌شان تست شدند (نه فقط نمونه)، و (۰۰۰۰ تا ۹۹۹۹): روی هر ۱۰۰۰۰ مقدار BCD ممکن

full_adder	(A, B, Cin)	8/8
adder4	$(A, B \in 0..15)$	256/256
adder5	$(A, B \in 0..31)$	1024/1024
cmp3	$(P, Q \in 0..3)$	16/16
eq5	$(A, B \in 0..31)$	1024/1024
mod3_checker	BCD	10000/10000
mod11_checker	BCD	10000/10000
main		40/40

جمع کل: ۱۰ از ۱۰ مجموعه تست پاس، صفر شکست. خروجی واقعی اجرای run_tests.sh (خلاصه شده، بدون شماره‌ی خط به خط بردارها):

```
circ : bcd_divisibility.circ
java : /opt/homebrew/opt/openjdk@21/bin/java
jar   : /Applications/Logisim-evolution.app/Contents/app/logisim-evolution-4.1.0-all.jar
vecs : 10

=== tests_adder4.vec → circuit 'adder4' ===
Loading test vector "tests_adder4..."vec
Running 256 ...vectors
Passed: 256, Failed: 0
PASS: tests_adder4.vec

=== tests_adder5.vec → circuit 'adder5' ===
Loading test vector "tests_adder5..."vec
Running 1024 ...vectors
Passed: 1024, Failed: 0
PASS: tests_adder5.vec

=== tests_cmp3.vec → circuit 'cmp3' ===
Loading test vector "tests_cmp3..."vec
Running 16 ...vectors
Passed: 16, Failed: 0
PASS: tests_cmp3.vec

=== tests_eq5.vec → circuit 'eq5' ===
Loading test vector "tests_eq5..."vec
Running 1024 ...vectors
Passed: 1024, Failed: 0
PASS: tests_eq5.vec

=== tests_full_adder.vec → circuit 'full_adder' ===
Loading test vector "tests_full_adder..."vec
Running 8 ...vectors
Passed: 8, Failed: 0
PASS: tests_full_adder.vec

=== tests_main.vec → circuit 'main' ===
Loading test vector "tests_main..."vec
Running 40 ...vectors
Passed: 40, Failed: 0
PASS: tests_main.vec

=== tests_mod11_checker_exhaustive.vec → circuit 'mod11_checker' ===
Loading test vector "tests_mod11_checker_exhaustive..."vec
Running 10000 ...vectors
Passed: 10000, Failed: 0
PASS: tests_mod11_checker_exhaustive.vec

=== tests_mod11_checker.vec → circuit 'mod11_checker' ===
Loading test vector "tests_mod11_checker..."vec
Running 35 ...vectors
Passed: 35, Failed: 0
```



```

PASS: tests_mod11_checker.vec

=== tests_mod3_checker_exhaustive.vec → circuit 'mod3_checker' ===
Loading test vector "tests_mod3_checker_exhaustive"...vec
Running 10000 ..vectors
Passed: 10000, Failed: 0
PASS: tests_mod3_checker_exhaustive.vec

=== tests_mod3_checker.vec → circuit 'mod3_checker' ===
Loading test vector "tests_mod3_checker"...vec
Running 35 ..vectors
Passed: 35, Failed: 0
PASS: tests_mod3_checker.vec

summary: 10/10 passed, 0 failed

```

چون هر دو مدار اصلی روی تمام فضای ورودی معتبر (نه فقط نمونه) تست شده‌اند، درستی مدار برای هر عدد BCD چهاررقمی تضمین شده است، نه فقط برای نمونه‌های امتحان شده.

نکات پیاده‌سازی

- **هندسه‌ی گیت‌ها دوبار تأیید شده:** یک بار با استخراج از یک مدار واقعی و کارکن، و یک بار مستقل با ساخت مدارهای پروب کوچک (یک NOT Gate تنها، یک AND+XOR دوتایی) و اجرای واقعی آن‌ها با -test-vector لاجیسیم - پیش از آنکه حتی یک خط از مدار اصلی نوشته شود. نتیجه: OR Gate/AND ورودی‌ها را در $(x-50, y\mp 20)$ می‌گذارد، XOR Gate در $(x-60, y\mp 20)$ ، NOT Gate در $(x-30, y)$ ؛ خروجی همه‌ی گیت‌ها دقیقاً روی مکان خودشان است.
- **پورت زیرمدار = افست از انکر:** برای هر بلوک، ظاهر سفارشی یک نگاشت $\text{pin} \rightarrow (x,y)$ دارد؛ وقتی بلوک در مکان (X,Y) جای می‌گیرد، آن پورت دقیقاً در $(X+x, Y+y)$ مدار بالادستی ظاهر می‌شود.
- **منطق تقسیم‌پذیری** («جمع ارقام mod 3»، «جمع متناوب ارقام mod 11») پیش از ساخت هر مداری با یک اسکریپت پایتون روی هر 10000 مقدار brute-force BCD تأیید شد؛ به همین دلیل وقتی بلوک‌های گیتی هم تک‌تک درست از آب درآمدند، ترکیب‌شان هم بی‌درنگ 10000/10000 پاس شد - بدون هیچ دور اشکال‌زدایی.